

22.10.25

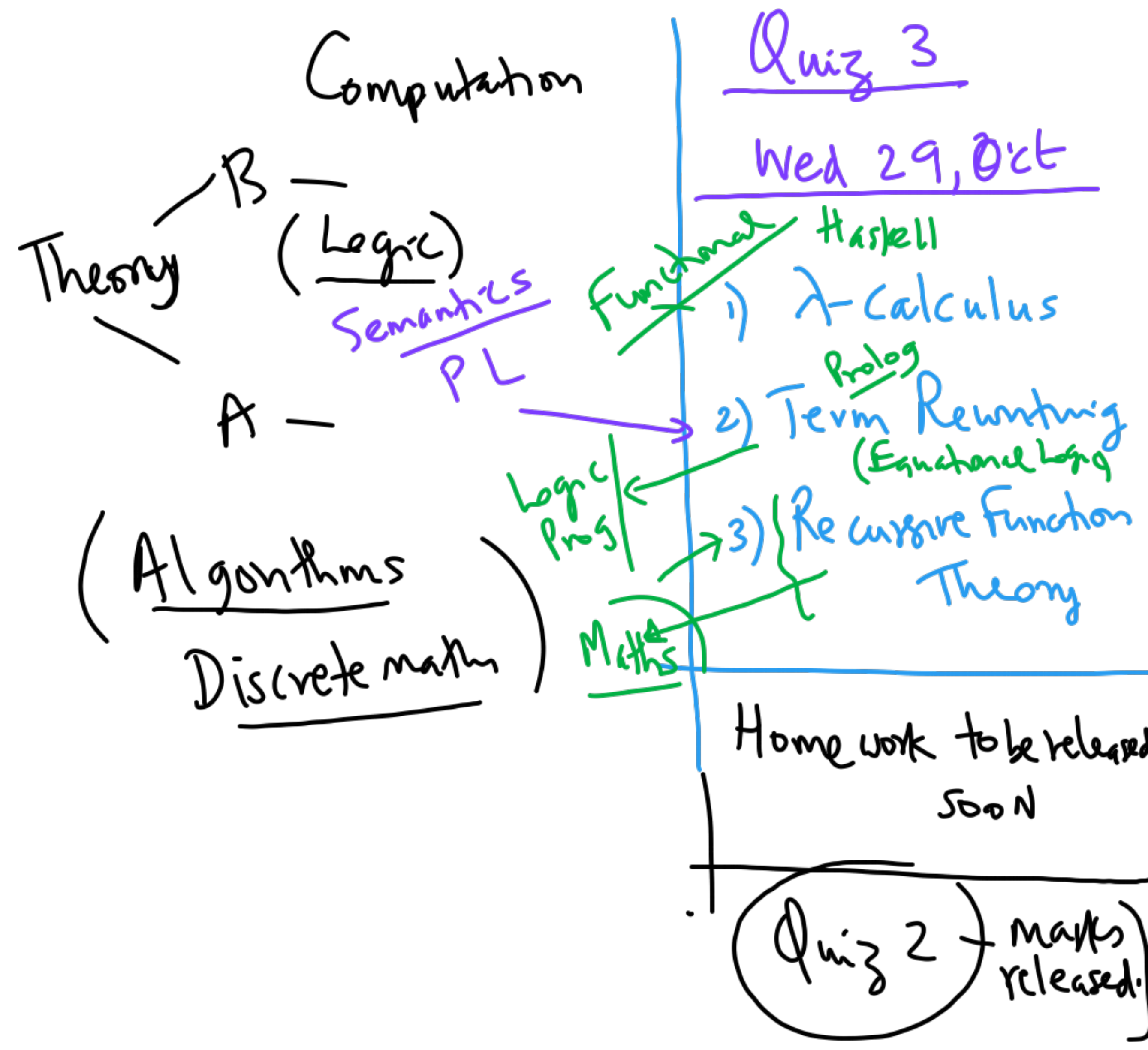
Rec. Fn. Theory



(Building block)

How to define functions f in a formal mathematical way

0
 $g(n)$ successor $\rightarrow n+1$



Basic Functions:

1. Zero $Z(n) = 0$
2. Succ $S(n) = n + 1$

3. Projection $P_i^k(n_1, \dots, n_k) = n_i$

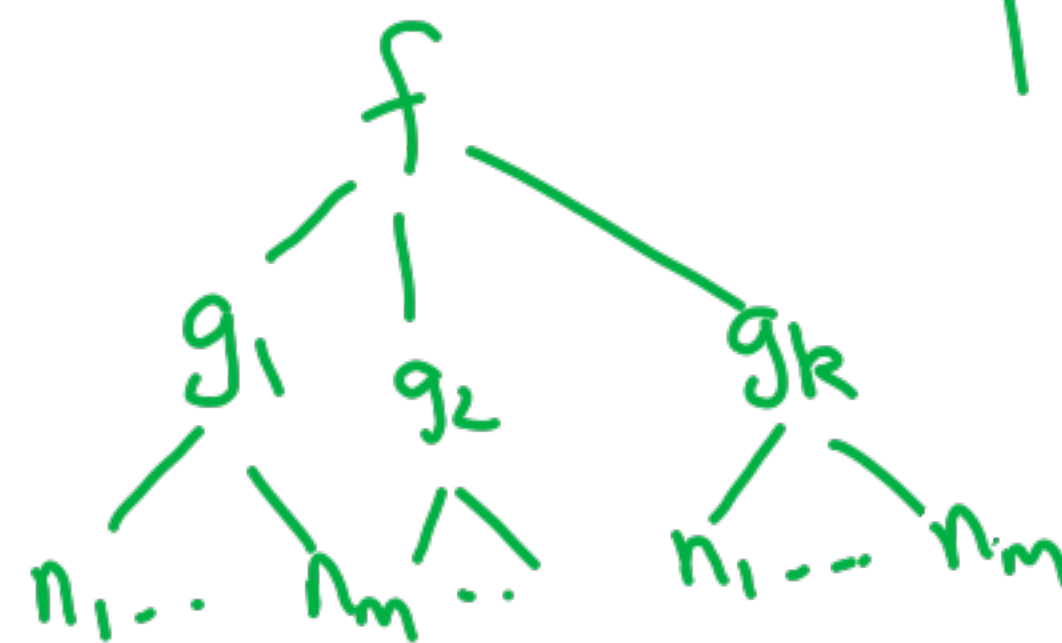
$k \geq i$
 $P_i^{32} \rightarrow \# \text{ inputs}$
 $P_i^7 \rightarrow \text{projected output}$

4. Composition

$f: \mathbb{N}^k \rightarrow \mathbb{N}$

$g_1, \dots, g_k: \mathbb{N}^m \rightarrow \mathbb{N}$

$f \circ (g_1, \dots, g_k) =$



Zero, Succ, Proj

Primitive Recursive Functions

given $g: N^k \rightarrow N$ $h: N^{k+2} \rightarrow N$

$f: N^{k+1} \rightarrow N$

define

Base Case →

$$0 \underset{\substack{| \\ n_1}}{\overset{f}{\swarrow}} \dots \underset{\substack{| \\ n_k}}{\searrow} = \underset{\substack{| \\ n_1}}{\overset{g}{\swarrow}} \dots \underset{\substack{| \\ n_k}}{\searrow}$$

Rec Case

$$s \underset{\substack{| \\ n}}{\overset{f}{\swarrow}} \dots \underset{\substack{| \\ n_k}}{\searrow} = \underset{\substack{| \\ n}}{\overset{h}{\swarrow}} \underset{\substack{| \\ f}}{\mid} \underset{\substack{| \\ n_1}}{\swarrow} \dots \underset{\substack{| \\ n_k}}{\searrow}$$

$$\underset{\substack{| \\ 0}}{\overset{\text{add}}{\swarrow}} \underset{\substack{| \\ n_2}}{\searrow} = \underset{\substack{| \\ n_2}}{\overset{P_1}{\mid}}$$

$$s \underset{\substack{| \\ n}}{\overset{\text{add}}{\swarrow}} \underset{\substack{| \\ n_2}}{\searrow} = \underset{\substack{| \\ n}}{\overset{h}{\swarrow}} \underset{\substack{| \\ \text{add}}}{\mid} \underset{\substack{| \\ n}}{\swarrow} \underset{\substack{| \\ n_2}}{\searrow}$$

where

$$\underset{\substack{| \\ x_1}}{\overset{h}{\swarrow}} \underset{\substack{| \\ x_2}}{\mid} \underset{\substack{| \\ x_3}}{\searrow} = s \underset{\substack{| \\ P_2^3}}{\overset{P_2^3}{\mid}} \underset{\substack{| \\ x_1}}{\swarrow} \underset{\substack{| \\ x_2}}{\mid} \underset{\substack{| \\ x_3}}{\searrow} \quad \checkmark$$

$$\begin{array}{c} \text{mult} \\ / \quad \backslash \\ 0 \quad n_2 \end{array} = \begin{array}{c} \text{Zero} \\ | \\ n_2 \end{array}$$

$$\begin{array}{c} \text{mult} \\ / \quad \backslash \\ s \quad n_2 \\ | \\ n \end{array} = \begin{array}{c} h \\ / \quad | \quad \backslash \\ n \quad \text{mult} \quad n_2 \\ \quad \wedge \\ \quad n \quad n_2 \end{array}$$

where

$$\begin{array}{c} h \\ / \quad | \quad \backslash \\ x_1 \quad x_2 \quad x_3 \end{array} = \begin{array}{c} \text{add} \\ / \quad \backslash \\ p_2^3 \quad p_3^3 \\ / \quad \backslash \quad \backslash \quad / \quad \backslash \\ x_1 \quad x_2 \quad x_3 \quad x_1 \quad x_2 \quad x_3 \end{array}$$

$\text{add}_0(p_2^3, p_3^3)$

H/w $\rightarrow$ monus $x \dot{-} y = \begin{cases} 0 & \text{if } x \leq y \\ x - y & \text{otherwise} \end{cases}$

$\searrow$ exp $\searrow$ fact $\searrow$ fib

Ackermann
fn.

$$a(3,3) \rightarrow 61$$

$$a(4,2) \rightarrow 2^{2^{16}} - 3$$

$$a(0, n) = n + 1$$

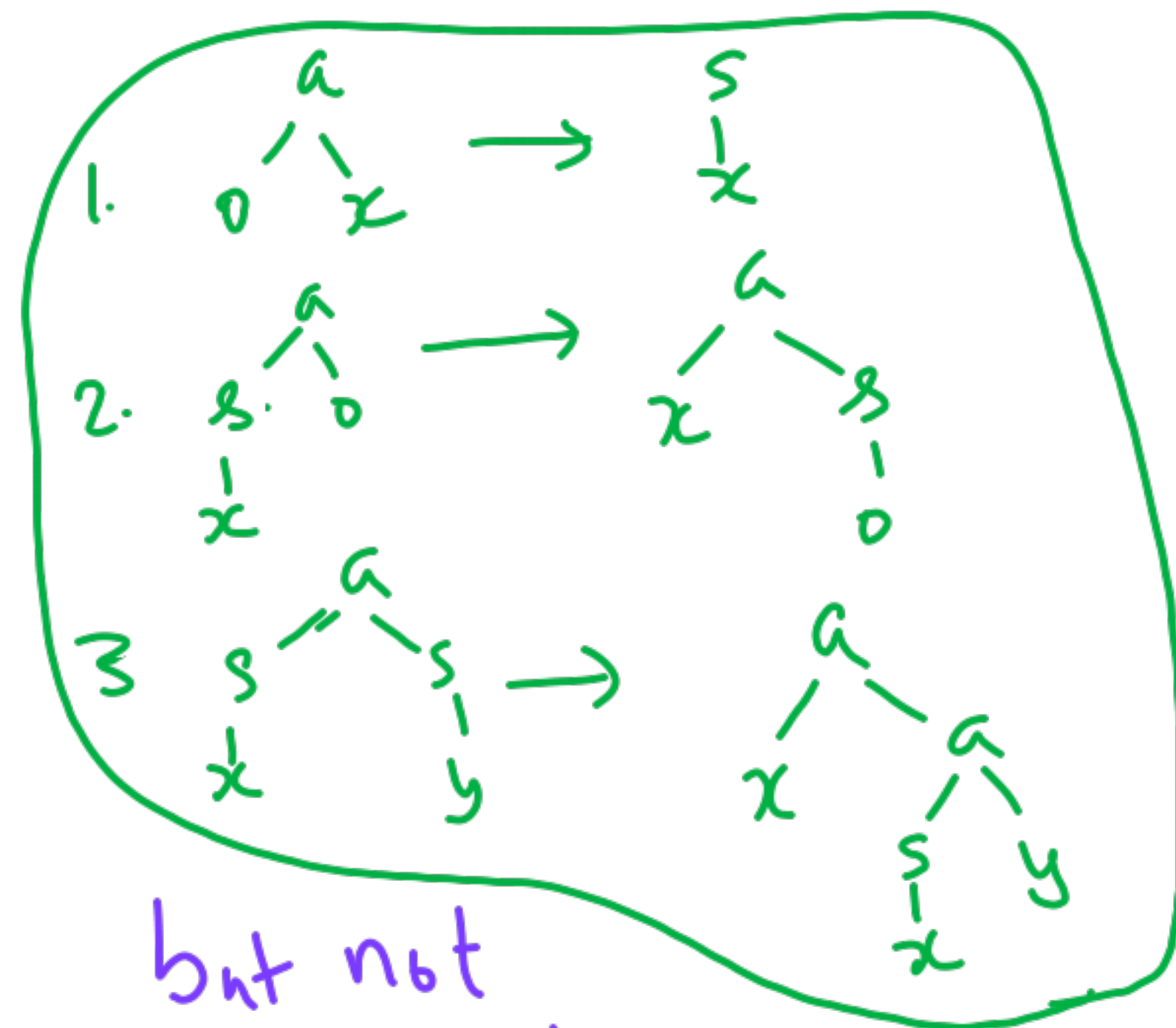
$$a(m, 0) = a(m-1, 1)$$

$$a(m, n) = a(m-1, a(m, n-1))$$

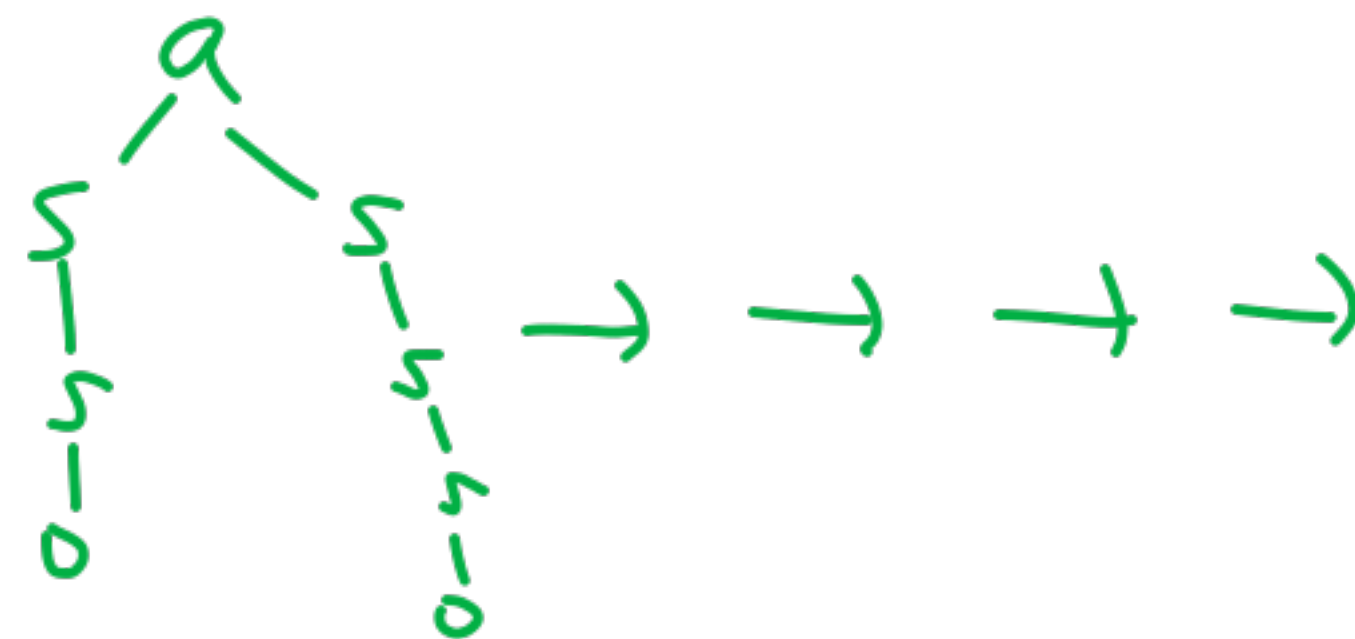
$$a(2,3) \rightarrow a(1, \underline{a(2,2)}) \rightarrow a(1, \underline{a(1, a(2,1))})$$

$$\rightarrow \rightarrow 9$$

Term Rewriting



but not
primitive recursion



Total
Partial
Functions

Minimization

Given

define

$$g : N^{k+1} \rightarrow N$$

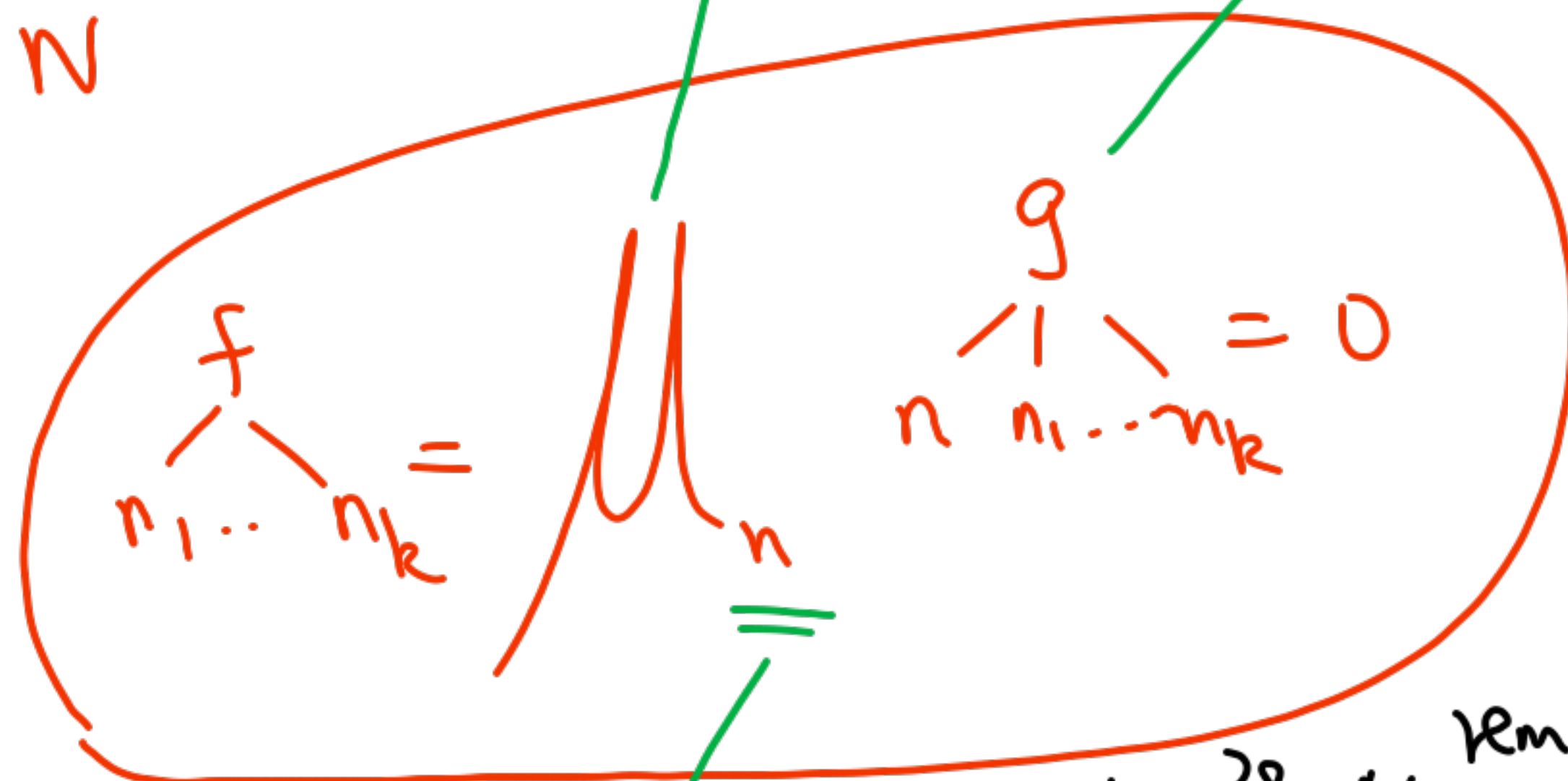
$$f : N^k \rightarrow N$$

Smallest
value of n
for which
becomes
0

using
minimization

$$f(13, 45)$$

$$= \begin{array}{c} 9 \\ \swarrow \downarrow \searrow \\ 0 \quad 13 \quad 45 \end{array} \begin{array}{c} 2 \\ \swarrow \downarrow \searrow \\ 1 \quad 13 \quad 45 \end{array} \rightarrow \begin{array}{c} 9 \\ \swarrow \downarrow \searrow \\ 2 \quad 13 \quad 45 \end{array} \begin{array}{c} 2 \\ \swarrow \downarrow \searrow \\ 2 \quad 13 \quad 45 \end{array}$$



$$f(n_1 \dots n_k) \equiv n$$

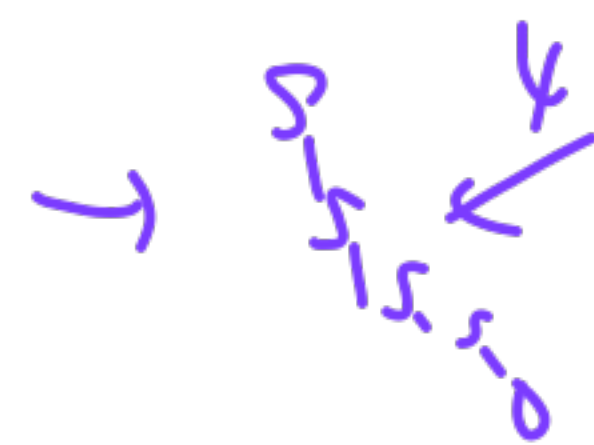
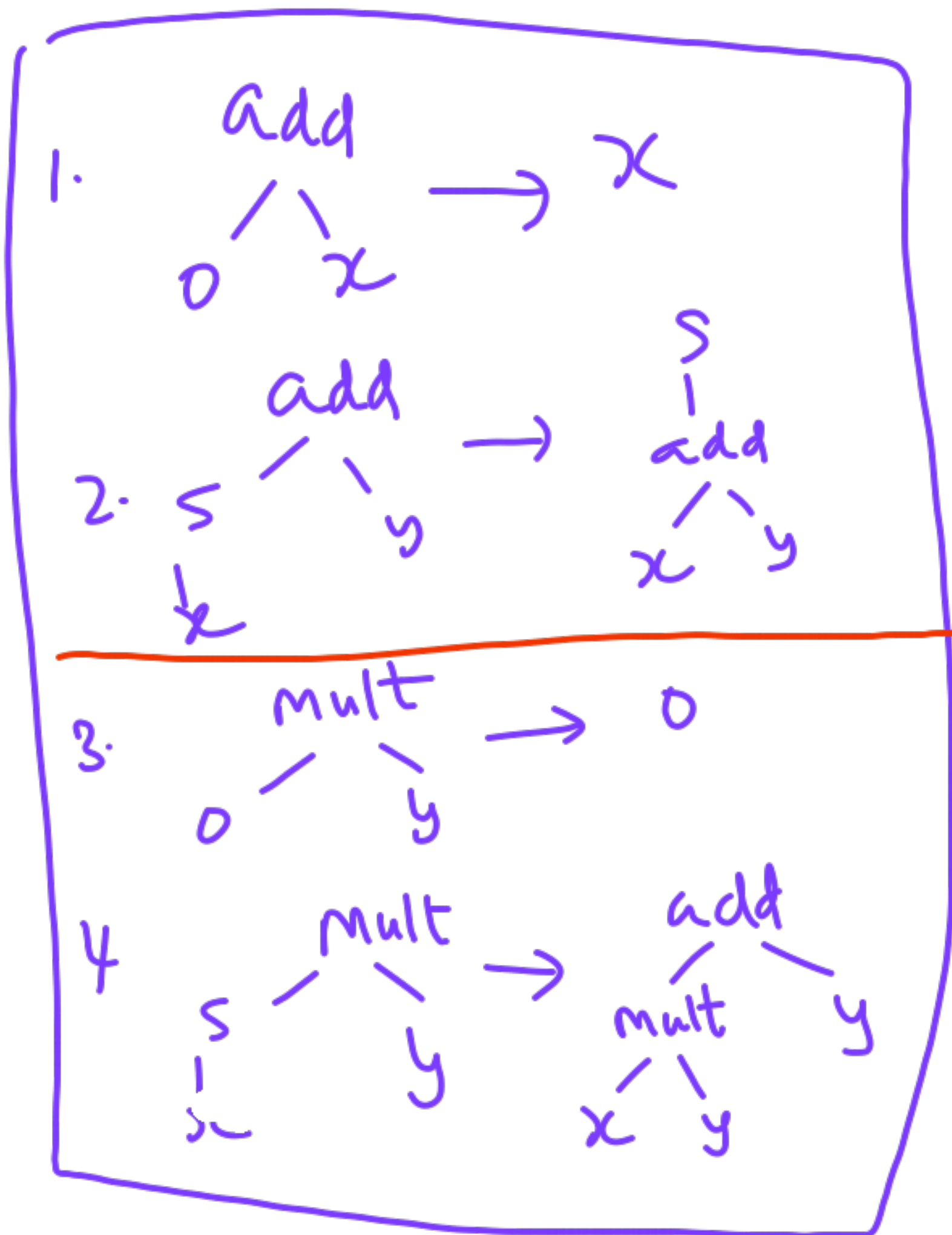
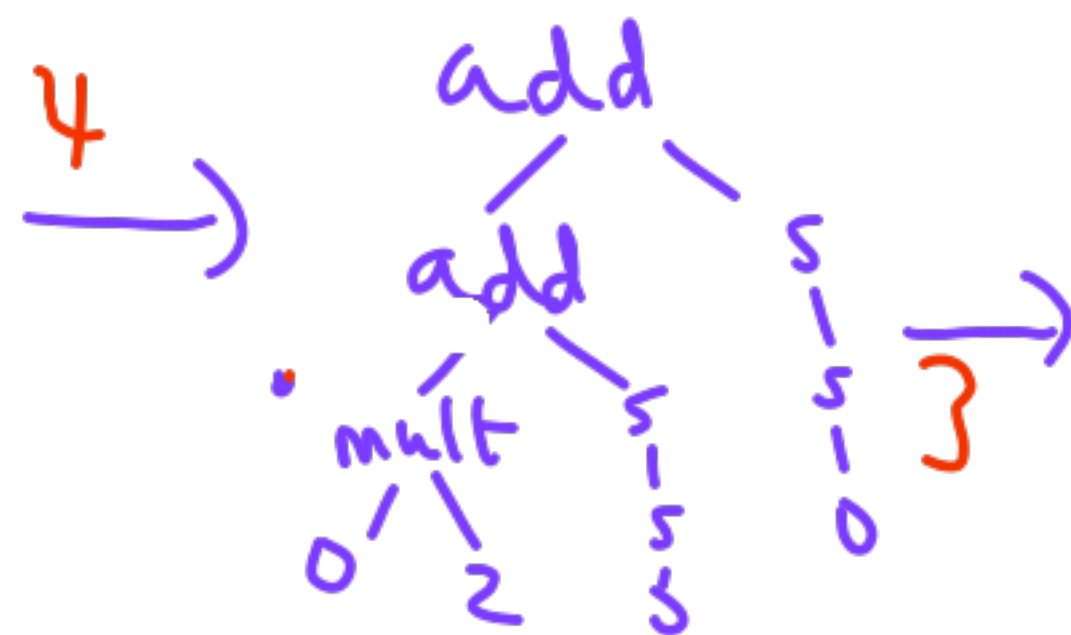
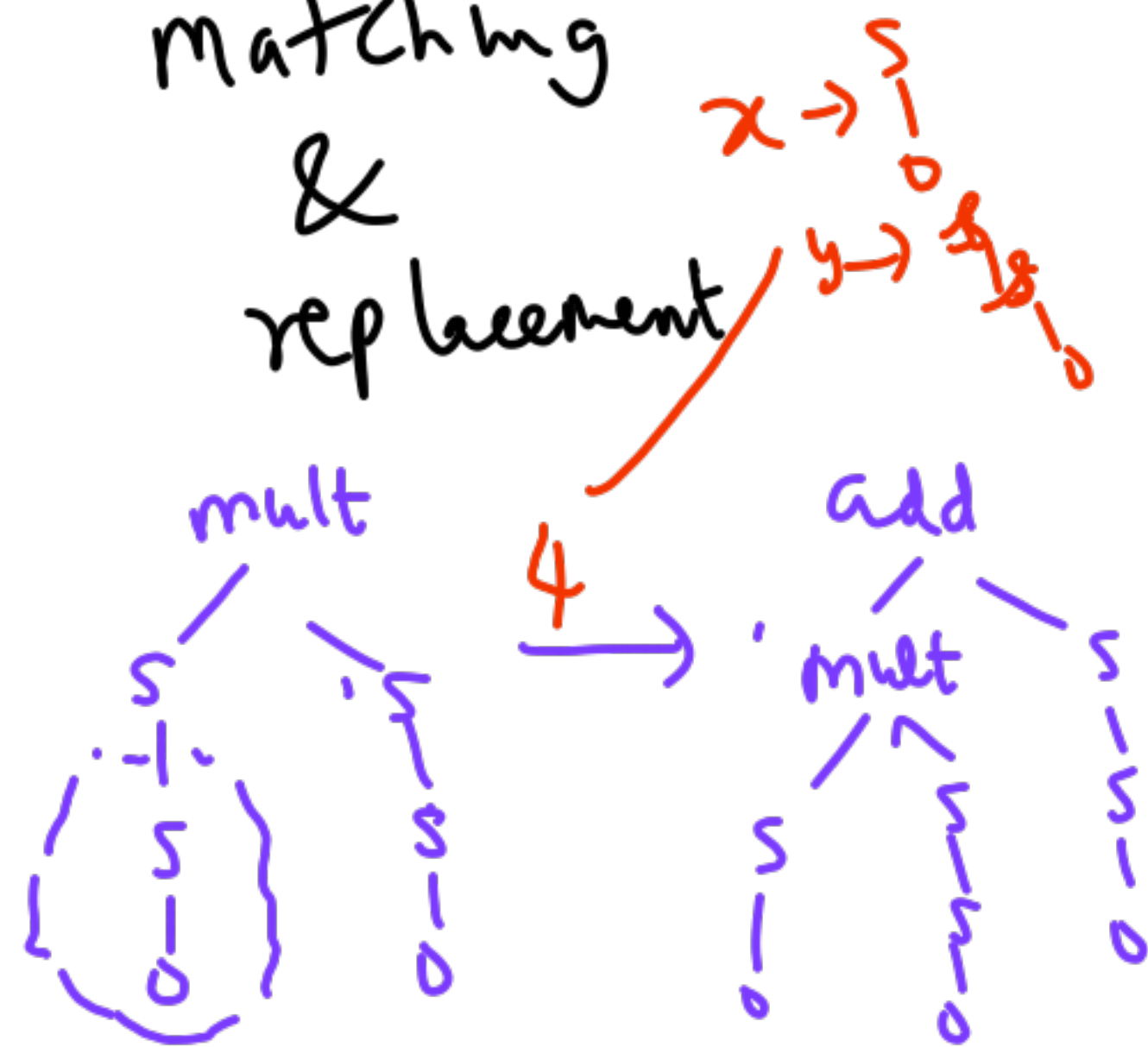
$$\begin{array}{l} H/w \rightarrow 38 \% 7 = 3 \text{ remainder} \\ 38 / 7 = 5 \text{ quotient} \end{array}$$

λ -calc Rec. fn theory

Term Rewriting

Ackerman

matching
&
replacement



make the rule apply

add

u v

?

s
s
0

u = s
u

~~s~~
add
u₁ v

=

~~s~~
s
0

u = s
0

v = s
0

1
u₁ = 0

v = s
0

2
u = s
s
0

v = 0

①

②

③